React JS

It is library to develop the frontend of a web application. Using React JS we can develop SPA. React JS is developed by facebook.

Old approach: html, css, js

Home.html: navbar: 30 lines html code footer: 50 line code

About.html: navbar: 30 line html code footer: 50 line code

Admissions.html: navbar : 30 line html code footer: 50 line code

50 html files

drawbacks:

1. No html code reusability
2. Not easy to maintain, not easy to scale
3. When user is visiting any of the page, the browser is getting a reload and due to this reload everytime browser will repaint the entire page. This is not efficient approach. Performance is the bottleneck.

SPA : Single Page Application

SPA will only have a single html file, and entire application will be displayed on that single html file only. When users are navigation between the links the browser will not be reloaded, and on the basis of url a new content will be dynamically displayed on that html page.

For SPA we have to break down the entire frontend into **components**.

Instead of creating multiple **pages**, we now have to create **components.**

Component: Ui element visible on the screen. Eg: button, input field, text, heading, table, link, image etc . Component can be at atomic level and the complete screen can also be referred as component.

For creating a component we have to create a **.js / .jsx** file

How to create a react application?

We will make use of a tool i.e. **vite** to generate/create a react application.

use this command to create a react app:

```
npx create-vite@latest
```

```
npm run dev
```

React application will have one already created component i.e. **App component** i.e in the src there might be a file **App.js / App.jsx. App** component is getting displayed first.

Entire react application gets displayed on the root level **index.html.**

How to create a component ?

Types of components:

1. Class component: To create this component we have to make use of the class syntax. Now a class is going to define the behavior or functionality of that component. We can make use of OOPs principles to create this type component. From the beginning of react js class components are there.
2. Function component: To represent the component if a function has been created then it is a function component. Function components are the light weight components.

Creating a class component:

Navbar.jsx:

```
import {Component} from "react";

export class Navbar extends Component{

        render(){
        // to provide something
        // render() will provide or display the ui for the cl
ass component
        return (
            <div>
```

```
                <a href="#">Home</a>
                <a href="#">About Us</a>
                <a href="#">Contact Us</a>
            </div>
        )
    }
}
// by this above inheritance we can say Navbar class will hav
e all the features to get recogniz as a ui component
```

Note: To make use of any component (either it is class component or function component) or we if we need to display that component we will get the tag name corresponding to the component name. Every tag will have opening and closing, either it is predefined html tag or it is custom tag for the component.

JSX: JavaScript and XML: .jsx extension is enabling a developer to write js code along with the html code.

Creating function component:

NavigationBar.jsx

```
export function NavigationBar(){
    return (
        <div>
            <a>..</a>
        </div>
    )
}
```

Note: JSX expressions must have one parent element

Features of a component:

1. Prop
2. State

These two features are going to be there in each and every type of component

Props: attributes/properties which are passed in the component

```
<p align="center">Hello</p>
<img src="" width="" height="">
<a href=""></a>
```

Props in a function component:

```
export function Intro(props) {

    const { headingText, description, buttonText } = props;

    return (
        <div>
            <h1>{headingText}</h1>
            <p>{description}</p>
            <button>{buttonText}</button>
        </div>
    )
}
```

```
<Intro headingText="" description="" buttonText="" ></Intro>
```

Note: React js will first prepare an object of all the props passed in the component

eg:

```
{
            headingText: "Welcome to our app",
            description: "This is our first react app",
            buttonText: "Click me"
          }
```

and the same object will be passed as argument of the function component.

The argument of function component will always receive the props only

A component should not change the value of its props, and it also cannot decide the value of its own props.

Props can be used to pass data from one component to another component.

Props in a class component:

For a class component props will become its property and we already have got a property from Component class due to inheritance and the name of that property is **“props”. Props now in a class are the member of a class**

```
this.props = {
            headingText: "Welcome to our app",
            description: "This is our first react app",
            buttonText: "Click me"
          }
```

```
export class Introduction extends Component{

    render(){
        console.log(this.props);
        const {headingText, description, buttonText} =  this.
props;
        return (
            <div>
                <h1>{headingText}</h1>
                <p>{description}</p>
                <button>{buttonText}</button>
            </div>
        )
    }
}
```

```
<Introduction headingText="" description="" buttonText="" ></
Intro>
```

Receiving props in:

1. Function component: **argument**
2. Class component: **class level member with inherited property i.e. this.props**

State:

State is such a type of property which a component can create by its own, can decide its value, and can change its value also

State is a native feature of a class component.

State in a class component:

state is class level member/property that is inherited from the Component class. Inside of the constructor we have to initialize it. We have to refer state on this keyword: this.state

```
import { Component } from "react";

export class Scoreboard extends Component{

    constructor(){
        super();
        this.state = {
            score : 0
        }
    }

    increaseScore = ()=>{
        this.setState({score: this.state.score + 3 });
    }

    decreaseScore = ()=>{
       this.setState({score: this.state.score - 1 });
    }

    render(){
        return (
```

```
            <div>
                <h1>Score: {this.state.score} </h1>
                <button onClick={this.increaseScore}>+</butto
n>
                <button onClick={this.decreaseScore}>-</butto
n>
            </div>
        )
    }

}
```

When state is updated/changed the component will be re-rendered

Virtual DOM: Copy of the actual browser DOM

Virtual DOM is faster

Virtual DOM → translated → Actual DOM

State in a function Component:

State is not a native feature of a function component.

Note: Not only the state, we can have certain more feature those were native to the class components only. Now in order to implement same feature in a function component react is providing the hooks.

Hook: hook is just a function.

Hook function naming convention: suggestion is that we should always have “use” prefix for the hook function name

To create a state in function component make use of: **useState()**

```
useState(initial-value); // will return two things
// 1. state value/data
// 2. function to update this state
// these two things will be returned in the form of array
const rs = useState(0); // rs is array
// rs[0] state value, data
// rs[1] function to update this state
```

```
// destructure the array returned by useState()
const [x,y] = useState(0); // x = state value, data & y= func
tion to update this state

// x and y are not readable names we must give a readable nam
e

const [score, setScore] = useState(0);
const [player, setPlayer] = useState("");
const [users, setUsers] = useState([]);
```

```
import { useState } from "react"

export function ScoreCard() {

    const [score, setScore] = useState(0);

    const increaseScore = () => {
        setScore(score + 3);
    }

    const decreaseScore = () => {
        setScore(score - 1);
    }

    return (
        <div>
            <h1>Score : {score}</h1>
            <button onClick={increaseScore}>+</button>
            <button onClick={decreaseScore}>-</button>
        </div>
    )
}
```

Lifecycle of a component:

There are certain functions already defined and they are going to be called automatically for each lifecycle stage of a component. Lifecycle functions are natively available in a class components, but for function component we need to make use of a hook.

Mounting phase : component is going to display on the screen

Unmounting phase: component is getting removed from the screen

Lifecycle in a class component:

constructor() : when a component will be mounted, first of all constructor will be called. It will be called only a single time during the entire lifecycle.

render(): When cons has finished the job, render will be called for the 1st time and it will provide the ui for that component. Now later on, if any state has been updated then **render()** will be called again.

componentDidMount(): It will be called only and only just after the first render call. Now if later render() function is getting executed due to state update then **componentDidMount() will not be called.** In general we can make use this lifecycle function to fetch data at the time of component load.

Rendering list in react component:

```
this.state = {
    myList : [10,20,30,40,50]
}

/*
    [<p>10</p>, <p>20</p>, ... ]
*/
render(){
    return (
        <div>
            {
                this.state.myList.map((value)=>{
                    return (
                        <p>{value}</p>
```

```
                    )
                })
            }
        </div>
    )
}

this.state = {
    myList = [
                    {id:111,name:"Arun"},
                    {id:112, name:"Aman"},
                    {id:113, name:"Suraj"}
    ]
}
/*
    [<div>
                    <h4>111</h4>
                    <p>Arun</p>
          </div>,
          <div>
                    <h4>111</h4>
                    <p>Arun</p>
          </div>,
          <div>
                    <h4>111</h4>
                    <p>Arun</p>
          </div>]
*/
render(){
    return (
        <div>

          {
              this.state.myList.map((value)=>{
                    return (
                        <div>
```

```
                                <h4>{value.id}</h4>
                                <p>{value.name}</p>
                            </div>
                        )
                    })
                }
            </div>
        )
    }
```